



University of Glasgow | School of
Computing Science

DayOfWeek DayOfMonth Month 2XXX

XX.XX am/pm – XX.XX am/pm

(Duration: 1 hour)

EXAMINATION FOR

DEGREES OF MSci, MEng, BEng, BSc, MA and MA (Social Sciences)

BD4 Exam Paper

Exam Rubric:

1. Answer All Questions
2. Total Marks: 60 (1 mark per minute)

INSTRUCTIONS TO INVIGILATORS

Please collect all exam question papers and return to School
together with exam answer scripts

Questions:

1. Briefly describe how HDFS deals with random accesses and any (dis)advantages.

Answer: random accesses are allowed using the routines `open()` and then `read()` / `seek()`. The problem is that what is read from the data node disk and communicated over the network is a huge block (eg 128MBs). So it is very coarse grained and thus costly.

[Total Marks: 3]

2. Assume an HBase configured to have tablets of size 64MB. What is the maximum size (in bytes) of the data that can be stored in this HBase?
Explain briefly.

Answer: The size of each row in the metadata tablet is 1KB. ACTUALLY ANY SIZE ASSUMED BY STUDENTS WILL BE ACCEPTED. As each tablet holds 64MB, each METADATA tablet then has 64K rows. So the ROOT tablet has 64K rows, each pointing to a METADATA tablet → there will be 64K METADATA tablets. Each row in a METADATA tablet has also 64K rows, each pointing to a tablet. So in total we have $64K * 64K$ tablets = $2^{16} * 2^{16}$ tablets = 2^{32} tablets. As each tablet holds 64KB, this yields $2^{32} * 2^{16}$ bytes = 2^{48} bytes. $\approx 256TBs$.

[Total Marks: 5]

3. What are the key design/architecture differences between HBase and Cassandra and their implications on system performance, scalability, and fault tolerance.?

Answer: Overall, the distribution paradigms are different: HBase uses a master-slave architecture, whereas Cassandra uses a completely decentralised architecture.

[2 marks]

Performance: The Master in HBase is more in danger of becoming a performance bottleneck. However, identifying a performance problem in Cassandra by monitoring, is more expensive to collect the global knowledge.

[1 mark]

Scalability: Cassandra makes scalability easier, as more nodes can be added gracefully and gradually to accommodate extra load.

[1 mark]

Fault Tolerance: HBase is susceptible to the single-point-of-failure problem for the Master. Fault tolerance w.r.t. failures of data nodes in Cassandra and HBase is very similar.

[2 marks]

[Total Marks: 6]

4. Given that failures can occur, explain in 2-3 sentences how HBase ensures that there will not be more than 1 entity playing the role of the Master.

Answer: Basically, there exists a unique exclusive lock MASTERLOCK maintained by Chubby. When a Master starts, it tries to lock this lock. If successful, it can be the only Master.

[Total Marks: 3]

5. Explain in 2-3 sentences how unassigned tablets are discovered in HBase.

Answer: Upon startup, the Master (after getting ex-lock MASTERLOCK) will receive from Chubby a list of the tablet servers. Then it will contact them and compile a list of all tablets they store, TABLET_LIST. Then master will scan METADATA tablets and see if any tablets in there are not in the TABLET_LIST.

[Total Marks: 3]

6. Consider a file that contains several documents. In the file, a document is stored in several lines, each of which contains the docID, followed by several words. For example, the file contents are like:

*doc-1 big data is a term for data sets that are massive and complex
doc-1 typically these data sets are analysed to derive knowledge
doc-2 machine learning is an area of artificial intelligence
doc-2 it is based on training statistical models and algorithms for
prediction/inference, etc.*

.....

You are asked to write MapReduce Java code that produces, for each word, a count of how many documents contain it.

Three classes **QuestionOne**, **QuestionOneMapper** and **QuestionOneReducer** are provided below. If necessary, you can add code in any of these classes and their methods. A fourth, **Parser**, class with the following methods is given: (1) `pr.parse(String)` is used to parse a line, (2) `pr.getID()` returns the docID of a line and (3) `pr.getTokens()` returns an ArrayList of all words in a line except docID.

```
public class QuestionOne extends Configured implements Tool
{
    public static void main(String[] args) {
        try
        {
            int response = ToolRunner.run(new QuestionOne(), args);
            if(response == 0)
            {
                System.out.println("QuestionOne finished successfully...");
            }
            else
            {
                System.out.println("QuestionOne failed.....");
            }
        }
        catch(Exception exception) ;
        {
            exception.printStackTrace();
        }
    }
}
```

```

@Override
public int run(String[] args) throws Exception {
    int result=0;
    Configuration configuration = getConf();
    Job job = Job.getInstance(configuration);
    job.setJobName("QuestionOne");
    FileInputFormat.addInputPath(job, new Path(args[0]));
    job.setMapperClass(QuestionOneMapper.class);
    FileOutputFormat.setOutputPath(job, new Path(args[1]));
    job.setJarByClass(getClass());
    job.setReducerClass(QuestionOneReducer.class);
    job.setInputFormatClass(TextInputFormat.class);
    job.setMapOutputKeyClass(Text.class);
    job.setMapOutputValueClass(Text.class);
    job.setOutputKeyClass(Text.class);
    job.setOutputValueClass(Text.class);
    job.waitForCompletion(true);
    return result;
}
}

public class QuestionOneMapper extends Mapper <LongWritable, Text, Text, Text>
{
    public void map(LongWritable key, Text value, Context context)
    {
        ....
    }
}

public class QuestionOneReducer extends Reducer<Text, Text, Text, Text >
{
    protected void reduce(Text key, Iterable<Text> values, Context context) throws
IOException, InterruptedException
    {
        ...
    }
}

```

Answer:

Class Mapper

method map

[5 Marks]

```

        pr.parse(value.toString);
        String docID = pr.getID();
        for(String word : pr.getTokens())
        {
            context.write(new Text (word), new Text(docID));
        }
    }

```

Class Reducer

method reduce

[5 Marks]

```
        HashSet<String> hs;
        int count = 0;
        for(Text value : values) {
            if (hs.add(value.toString())) count++;
        }
        context.write(key, new IntWritable(count));
```

The most efficient solution would be to use a **combiner** function at the mapper side, which is identical to the reducer function. So in configuration add:

```
job.setCombinerClass(QuestionOneReducer.class);
```

[5 Marks]

[Total Marks: 15]

7. Assume a big, 2-dimensional dataset of points (with x- y-coordinates) is given like:

```
2.909,  5.987
34.87,  9.297
98.65,  12.76
982.98, 218.87
....
```

Given a query point (e.g 3.9, 8.78) as a parameter from a user, you are asked to write the Mapreduce Java code that calculates the *nearest 10 points* to the given query point.

You are provided with three classes QuestionTwo, QuestionTwoMapper, QuestionTwoReducer below to help in this task. **Add a (detailed pseudocode or) Java code in the three classes where/if needed.** A parser class, Parser, is also provided that has the following methods:

- A. public Point parse(String input): returns a data type "Point" for a given file input line (e.g. 2.909, 5.987) of the dataset;
- B. public double calDistance(Point a, Point b): returns the distance between points a and b
- C. public boolean add(String c, double dis): adds c in an arrayList, which can only contain 10 elements sorted on distance, in ascending order, deleting from a full arrayList the point with the maximum distance.
- D. public ArrayList getStoredData(): returns the arrayList that contains 10 data elements along with their distance.
- E. public double getDistance(int i) : returns the distance of an element stored at the ith index in the ArrayList mentioned at (C) and (D) above.
- F. public boolean isFull(): returns true if there are 10 elements in the Arraylist.
- G. public int size(): returns the number of elements in the arraylist.

```
public class QuestionTwo extends Configured implements Tool
{
    public static void main(String[] args) {
        try
        {
            int response = ToolRunner.run(new QuestionOne(), args);
```

```

        if(response == 0)
        {
            System.out.println("QuestionOne finishes succesfully...");
        }
        else
        {
            System.out.println("QuestionOne failed.....");
        }
    }
    catch(Exception exception)
    {
        exception.printStackTrace();
    }
}

```

@Override

```

public int run(String[] args) throws Exception {
    int result=0;
    Configuration configuration = getConf();
    Job job = Job.getInstance(configuration);

    configuration.set("parameter", args[2]);

    job.setJobName("QuestionTwo");
    FileInputFormat.addInputPath(job, new Path(args[0]));
    job.setMapperClass(QuestionTwoMapper.class);
    FileOutputFormat.setOutputPath(job, new Path(args[1]));
    job.setJarByClass(getClass());
    job.setReducerClass(QuestionTwoReducer.class);
    job.setInputFormatClass(TextInputFormat.class);
    job.setMapOutputKeyClass(Text.class);
    job.setMapOutputValueClass(DoubleWritable.class);
    job.setOutputKeyClass(NullWritable.class);
    job.setOutputValueClass(Text.class);
    job.waitForCompletion(true);
    return result;
}
}
public class QuestionTwoMapper extends Mapper <LongWritable, Text, Text,
DoubleWritable>
{
    ...
}

```

```

public class QuestionTwoReducer extends Reducer<Text, DoubleWritable,
NullWritable, Text >
{
    ...
}

```

Answer:method **run()**:

\\ set number of reducer to 1.

Configuration.set("numberofreducertasks", 1);

[2 Marks]**Mapper class**

\\ declare point qry

Point qry;

setup(Context context) Method:**[2 Marks]**

\\ get the parameter to a mapper

strqry = context.getConfiguration().get("qryPoint");

\\ initialise parameter to Point

qry = parser.parse(parameter);

map (LongWritable key, Text value, Context context) method:**[5 Marks]**

\\ parse the input line

Point b = parser.parse(value.toString());

\\ calculate distance

double dis= parser.calDistance(b, qry);

\\ add To 10-NN list if this line has lesser distance ;

if ((pr.getdistance(pr.size() - 1) > val.get() || ! pr.isFull())

pr.add(value.toString(), dis);

cleanup (Context context):**[4 Marks]**

\\ for point in kNN List

for Point point : parser.getStoredData()

\\ emit to reducer

context.write(point.getSting(), point.getDistance);

Reducer:

\\ declare parser

Parser pr;

setup(Context context) Method:**[2 Marks]**

\\ initialise parser

Parser pr = new Parser();

reduce() method:**[5 Marks]**

\\ add To KNN list if this line has lesser distance

for(Text val : values)

if ((pr.getdistance(pr.size() - 1) > val.get() || ! pr.isFull())

pr.add(key.toString(), val.get());

cleanup (Context context):**[5 Marks]**

\\ for point in kNN List

for Point point : parser.getStoredData()

\\ write to a file

context.write(NullWritable.get(), point.getSting()) ;

[Total Marks: 25]